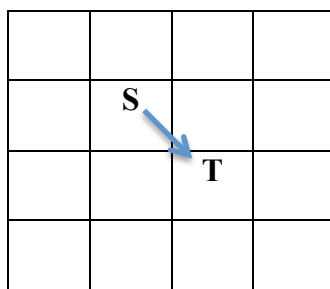


Bilkent University
CS 473 / 573 Midweek Exam III

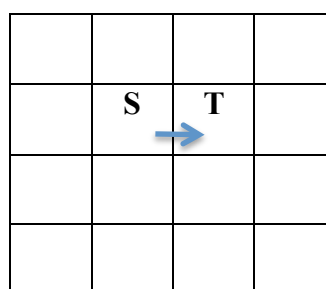
Duration: 30 minutes

Q2 (40 pts)

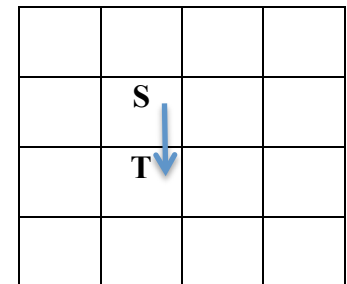
For a given city with $n \times n$ districts, there are at most five outgoing bus routes from each district. These five routes are shown for a specific district ($D[2][2]$) in a sample city with 4×4 districts, where **S** denotes the source district and **T** denotes the target district.



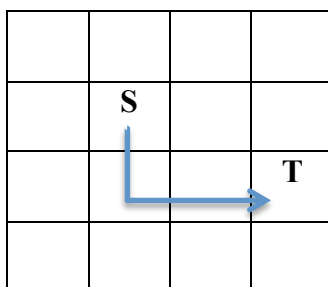
Route 1



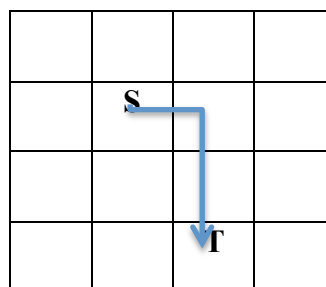
Route 2



Route 3



Route 4



Route 5

As seen in the above figures, the bus routes from a district only goes to the rightwards and downwards districts. Starting from the top left corner district ($D[1][1]$), give an $O(n^2)$ -time **dynamic programming** algorithm that computes the **number of possible ways** to reach the bottom right corner district ($D[n][n]$). In your formulation, let $m_{i,j}$ denote the number of possible ways to reach district $D[i][j]$ from $D[1][1]$.

Clearly show the second and third steps of your dynamic programming formulation;

Step 2: Recursively define the number of possible ways to reach a district, i.e., $m_{i,j}$

Step 3: In pseudocode, an $O(n^2)$ -time dynamic programming algorithm that computes and returns the total number of ways to reach the bottom right corner district

Note that you are not asked to list all the possible ways, but only the number of them.

CHEATSHEET

Dynamic Programming

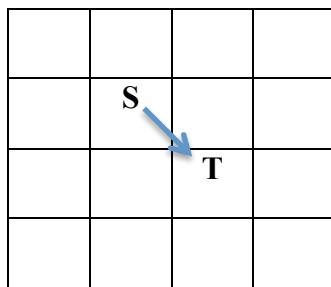
- **Step 1:** Characterize the structure of an optimal solution. An optimal solution to an instance of the problem contains optimal solutions to subproblem instances.
- **Step 2:** Recursively define the value of an optimal solution in terms of the optimal solutions to the subproblems.
- **Step 3:** Compute the value of an optimal solution, typically in a bottom-up fashion.
- **Step 4:** Construct an optimal solution from the computed information.

Bilkent University
CS 473 / 573 Midweek Exam III

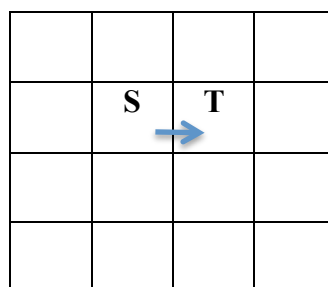
Duration: 20 minutes

Q3 (25 pts)

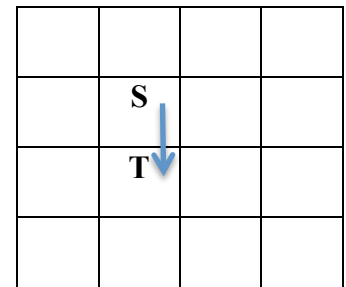
The setup of this question is same as Q2. That is, for a given city with $n \times n$ districts, there are at most five outgoing bus routes from each district. These five routes are shown for a specific district ($D[2][2]$) in a sample city with 4×4 districts, where **S** denotes the source district and **T** denotes the target district.



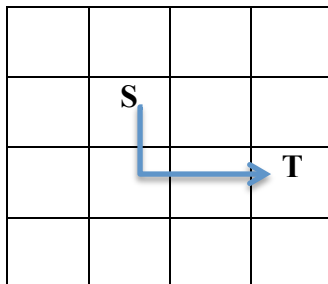
Route 1



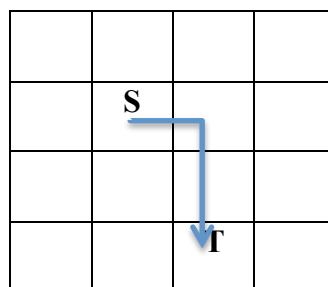
Route 2



Route 3



Route 4



Route 5

As seen in the above figures, the bus routes from a district only goes to the rightwards and downwards districts. Starting from the top left corner district ($D[1][1]$), give an $O(n^2)$ -time **dynamic programming** algorithm that computes the **number of possible ways** to reach the bottom right corner district ($D[n][n]$) **with the following constraint**;

- Same route cannot be taken in two successive route selections. For example, if you select Route i from going **S** to **T**, when leaving **T** you must select a route different than Route i .

You should clearly state the difference in your solution to address the given constraint in this variant of the question. You are only asked to clearly **define the table(s)** to be used for an $O(n^2)$ -time dynamic programming algorithm and provide the **recursive formulation**. You don't have to give an implementation in pseudocode.

Note that you are not asked to list all the possible ways, but only the number of them.

CHEATSHEET

Dynamic Programming

- **Step 1:** Characterize the structure of an optimal solution. An optimal solution to an instance of the problem contains optimal solutions to subproblem instances.
- **Step 2:** Recursively define the value of an optimal solution in terms of the optimal solutions to the subproblems.
- **Step 3:** Compute the value of an optimal solution, typically in a bottom-up fashion.
- **Step 4:** Construct an optimal solution from the computed information.

Q3: [40min] (45pts) In a palindromic subsequence, elements read the same backward and forward. A subsequence is a sequence that can be derived from another sequence by deleting some or no elements without changing the order of the remaining elements. Longest Palindromic Subsequence (or LPS) of a given sequence $X = \langle x_1, x_2, \dots, x_n \rangle$ is the longest of all palindromic subsequences of X . In other words, given a sequence $X = \langle x_1, x_2, \dots, x_n \rangle$, we would like to find the length of its Longest Palindromic Subsequence (or LPS) $Y = \langle y_1, y_2, \dots, y_m \rangle$, where indices of Y $\langle i_1, i_2, \dots, i_m \rangle$ match indices of X $\langle j_1, j_2, \dots, j_m \rangle$ with $1 \leq j_1 \leq j_2 \leq \dots \leq j_m \leq n$ and $y_{j_1} = x_{j_1}, y_{j_2} = x_{j_2}, \dots$. As an example, for a sequence $X = \langle a, a, b, c, d, e, b, a, f \rangle$, a LPS of X is $Y_1 = \langle a, b, d, b, a \rangle$ with length 5. Another LPS of X is $Y_2 = \langle a, b, e, b, a \rangle$.

Notice that if beginning and end elements are the same ($x_1 = x_n$) in a sequence $X = \langle x_1, x_2, \dots, x_n \rangle$, then one can find a LPS of $\langle x_2, x_3, \dots, x_{n-1} \rangle$ and use this solution to the subproblem in solving the original one. Similarly, in case they are not the same ($x_1 \neq x_n$), then a LPS of X is either equal to that of $\langle x_1, x_2, \dots, x_{n-1} \rangle$ or that of $\langle x_2, x_3, \dots, x_n \rangle$, whichever might be longer. For this problem,

- a. (5pts) First, recursively define the value of an optimal solution:

$LPS(X_{ij}) = LPS(X_{i+1,j-1}) + 2$	if $i < j$ and $x_i = x_j$
$\max(LPS(X_{ij-1}), LPS(X_{i+1,j}))$	if $i < j$ and $x_i \neq x_j$
0	if $i > j$
1	if $i = j$

- b. (15pts) Now, write a divide-and-conquer algorithm for computing the value of an optimal solution. Discuss if this solution is efficient after analyzing the worst-case running time of your algorithm:

```
int findLPSLength(X, p, r)
    if p > r return 0
    if p == r return 1
    if X[p] == X[r]
        return 2 + findLPSLength(X, p+1, r-1)
    l1 = findLPSLength(X, p+1, r)
    l2 = findLPSLength(X, p, r-1)
    return max(l1, l2)
```

Recursion tree analysis yields that the algorithm's worst case running time is lower bounded by an exponential function; hence, the algorithm is clearly inefficient.

- d. (5pts) Finally, write an algorithm for outputting the corresponding LPS using the output of the algorithm in the previous part:

I

Final Exam Sample Solutions, 4 Sessions, May 17, 2021

```
int findLPS(X, n, m) // runs in O(n) time
LPS = "" // is to maintain the second half of LPS
i=1, j=n
while i < j do
    if X[i] = X[j]
        LPS = LPS + X[i]
        i++, j--
    else if m[i,j-1] > m[i+1,j]
        j--
    else
        i++
if i = j
    return reverse(LPS) + X[i] + LPS
else
    return reverse(LPS) + LPS
```

Make sure to analyze the running time of your algorithms.

- c. (20pts) Then, write an **efficient** algorithm for computing the length of an LPS in a bottom up manner (**no memoized solutions**) by maintaining a table for each sub-problem needed to directly or indirectly solve the original one:

```
int findLPSLength(X, n) // runs in  $O(n^2)$  time
    for i=1 to n do
        m[i,i-1] = 0
        m[i,i] = 1
    for l=2 to n do
        for i=1 to n-l+1 do
            j=i+l-1
            if X[i] = X[j]
                m[i,j] = m[i+1,j-1]+2
            else
                m[i,j] = max(m[i,j-1],m[i+1,j])
    return m[1,n]
```

BILKENT UNIVERSITY
CS473 / 573 Midweek Exam 5

Duration: 45 minutes

cs473atbilkent@gmail.com, UD: 533 364 9456, EB101: 552 289 9530, EB102: 555 418 9178, EB103: 537 519 6542

Q1 (50 Points)

Let $X = x_1x_2x_3\dots x_m$ and $Y = y_1y_2y_3\dots y_n$ be two strings of lengths m and n respectively. An alignment is an assignment of gaps between X and Y 's letters so that each letter in one string corresponds to either a gap or a letter in the other sequence. For aligning the sequences, you can either pass a mismatching letter or introduce a gap to either string. However, these operations incur a penalty :

- A penalty of p_{xy} is incurred for mismatching characters.
- A penalty of p_{gap} is incurred for each gap on either string. A gap is represented by '_' character.

Given strings X and Y and penalty costs p_{xy} and p_{gap} , you are asked to find the alignments of X and Y with a **minimum** penalty.

Example: For strings $X = ACCCGA$, $Y = ACCGT$ and penalties $p_{gap} = 3$, $p_{xy} = 2$, an optimal alignment would be $X = ACCCGA$ and $Y = A_CCGT$ with a penalty of 5 (3 for the gap inserted in Y and 2 for mismatching characters A and T at the end of sequences).

For a **dynamic programming** solution:

- Recursively define the value of an optimal solution.
- Write a pseudocode for an algorithm (**no memoized solutions**) that computes and returns the minimum possible penalty.
- Analyze the worst-case time complexity of your algorithm.

Let $DP[i][j]$ be the penalty of optimal alignment of X_i and Y_j ,
 in which X_i is the first i letters of X and Y_j is the first j letters of Y .

Then,

$$DP[i][j] = \begin{cases} i * \text{pgap} & \text{if } j = 0 \\ j * \text{pgap} & \text{else if } i = 0 \\ DP[i-1][j-1] & \text{else if } x[i] == y[j] \\ \min(DP[i-1][j-1] + p_{xy}, DP[i-1][j] + \text{pgap}, DP[i][j-1] + \text{pgap}) & \text{else} \end{cases}$$

Total min. penalty is $DP[m][n]$

min-penalty ($x, y, p_{xy}, \text{pgap}$)

$m \leftarrow x.\text{length}$

$n \leftarrow y.\text{length}$

create array $dp[m+1][n+1]$

$O(m)$ $\left\{ \begin{array}{l} \text{for } i \leftarrow 0 \text{ to } m: \\ \quad dp[i][0] = i * \text{pgap} \end{array} \right\}$
 $O(n)$ $\left\{ \begin{array}{l} \text{for } j \leftarrow 0 \text{ to } n: \\ \quad dp[0][j] = j * \text{pgap} \end{array} \right\}$

fill top row and leftmost column

$m \leftarrow x.length$

$n \leftarrow y.length$

create array $dp[m+1][n+1]$

$O(m)$ $\left\{ \begin{array}{l} \text{for } i \leftarrow 0 \text{ to } m: \\ \quad dp[i][0] = i * pgap \end{array} \right\}$

$O(n)$ $\left\{ \begin{array}{l} \text{for } j \leftarrow 0 \text{ to } n: \\ \quad dp[0][j] = j * pgap \end{array} \right\}$

fill top row and leftmost column

$O(mn)$ $\left[\begin{array}{l} \text{for } i \leftarrow 1 \text{ to } m \\ \quad \text{for } j \leftarrow 1 \text{ to } n \\ \quad \quad \text{if } x[i] = y[j] \text{ // if char's are same, incur no penalty} \\ \quad \quad \quad dp[i][j] = dp[i-1][j-1] \\ \quad \quad \text{else} \\ \quad \quad \quad dp[i][j] = \min(dp[i-1][j-1] + pxy, dp[i-1][j] + pgap, dp[i][j-1] + pgap) \end{array} \right.$

return $dp[m][n]$

Time complexity: $O(mn)$

Q1. (60 points, 45 minutes)

As an avid NBA fan, you have decided to play a simplified form of YAHOO fantasy basketball game with friends for the 2019-2020 season.

In this version of fantasy basketball, each participant is given 100 credits and selects 5 NBA players prior to the start of the season. Each NBA player has a price that is calculated according to their form throughout their careers. Players earn a score for their performance in each game they play. A participant’s score is the sum of the scores of players they have selected. After the end of the season, participant with highest score wins.

After the season ends, you have 2 questions in your mind. First one is what would be the maximum number of scores that can be earned if you could select as much players as you want with 100 credits. Secondly, you wonder what would be the best team, i.e., which combination of 5 players would earn the maximum score. For these questions, you think you can use dynamic programming. You are given an array $P[1...n]$, each $P[i]$ is in the 3-tuple form ($\langle$ Player Name, price, score $\rangle$).

Example:

Assume that you are given a list of 11 basketball players in the table below:

Player Name	Price	Score
Tony Parker	15	120
Manu Ginobili	10	90
Rajon Rondo	10	55
Kobe Bryant	20	220
Damian Lillard	10	60
Tim Duncan	25	250
David Robinson	20	120
Lebron James	27	300
Charles Barkley	15	150
Kevin Garnett	10	80
Joakim Noah	3	20

To answer the first question, selecting Kobe Bryant, Manu Ginobili, Tim Duncan, Lebron James, Charles Barkley and Joakim Noah, a total of 6 players, costs $20+10+25+27+15+3=100$ credits which yields the maximum score $220+90+250+300+150+20=1030$.

To answer the second question, selecting Kobe Bryant, Manu Ginobili, Tim Duncan, Lebron James and Charles Barkley costs $20+10+25+27+15=97$ credits which yields the maximum score $220+90+250+300+150=1010$. Note that optimal solution can be achieved by spending less than 100 credits.

Q1a. (30 points)

For a **dynamic programming** solution:

- Recursively define the maximum score $S_{i,j}$ that can be obtained by using the first i players and j credits.
- Write the pseudocode for the algorithm that computes and returns the maximum score that can be obtained by using at most 100 credits.

Q1b. (30 points)

For a **dynamic programming** solution:

- Recursively define the maximum score $S_{i,j,k}$ that can be obtained by selecting exactly k players from first i players using j credits.
- Write the pseudocode for the algorithm that computes and returns the maximum score that can be obtained by using at most 100 credits and selecting exactly 5 players.

CHEAT SHEET

Dynamic Programming

Step 1: Characterize the structure of an optimal solution. An optimal solution to an instance of the problem contains optimal solutions to subproblem instances.

Step 2: Recursively define the value of an optimal solution in terms of the optimal solutions to the subproblems.

Step 3: Compute the value of an optimal solution, typically in a bottom-up fashion.

Step 4: Construct an optimal solution from the computed information.

Example: Fully Parenthesizing Matrix-chain Multiplication

Step 1: Given an optimal parenthesization $(A_1A_2...A_k)(A_{k+1}A_{k+2}...A_n)$, the parenthesization of the subchain $A_1A_2...A_k$ and the parenthesization of $A_{k+1}A_{k+2}...A_n$ should be optimal.

Step 2: m_{ij} : minimum number of scalar multiply-add operations needed to compute the subchain product $A_iA_{i+1}...A_j$

$$m_{ij} = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \{m_{ik} + m_{k+1,j} + p_{i-1}p_kp_j\} & \text{if } i < j \end{cases}$$

Bilkent University, CS 473/573, Session 4, 32pts, 40min

Q4: You are to organize a canoe trip down the river. Canoes are available for rent at a sequence of trading posts along the river. You will start your trip by renting a canoe at post 1 (where the canoeing begins) and end the trip at post n (where the canoeing ends). You are allowed to stop at any post, drop off the canoe you have and rent another one. For all pairs (i, j) with $i < j \leq n$, the cost of renting a canoe at post i and dropping it off at post j is provided. For example, if there are four posts in total, the costs might be:

$R[i, j]$	$j = 1$	2	3	4
$i = 1$	-	10	15	50
2	-	-	40	20
3	-	-	-	35
4	-	-	-	-

Your job is to design an **efficient** algorithm to plan the sequence of canoe rentals to **minimize the total cost**. In the example shown, an optimal solution is to rent a canoe from post 1 to post 2 (cost 10), then rent a canoe from post 2 to post 4 (cost 20) with a total cost of 30. Your grade will be based on the efficiency of your algorithm.

Bilkent University, CS 473/573, Session 2, 50pts, 45min

Question 2: Imagine a climber trying to climb on top of a wall. A wall is constructed out of square blocks of equal size, each of which provides one handhold. Some handholds are more dangerous than others. Initially (while on the ground), the climber can start from any one of the blocks in the first row. Once off the ground, from each block the climber is currently on, s/he can reach three blocks of the row just above: the one directly above, the one towards upper right, and the one towards upper left (unless right or left are not available because it is the end of the wall). The goal is to find the least dangerous path from the bottom of the wall to the top, where the danger rating (cost) of a path is the sum of danger ratings (costs) of blocks used on that path.

The problem is presented as follows. The input is an $m \times n$ grid, given as a 2D array $D = (d_{ij})$, where each block has a positive cost d_{ij} associated with it, where the higher the cost, the more dangerous the block. The index of bottom row is **one** and the index of top row is m . From block (i, j) , $i < m$, you can reach blocks

- $(i + 1, j - 1)$ if $j > 1$,
- $(i + 1, j)$, or
- $(i + 1, j + 1)$ if $j < n$

Below is an example of an input block grid, where $m = 4$ and $n = 5$. A least dangerous path is highlighted, which has a total cost of **12**:

2	8	9	5	8
4	4	6	2	3
5	7	5	6	1
3	2	5	4	8

Towards a **dynamic programming algorithm** to find the cost of the least dangerous path:

- a. Give the **recursive definition** for the value of an optimal solution (**Step 2** of dynamic programming formulation).
- b. Write the pseudo-code for the bottom-up algorithm that computes the **cost** of the least dangerous path (**Step 3** of dynamic programming formulation). Memoization-based solutions will be heavily penalized.

2) (60 pts) In this problem, N units of a resource are to be allocated to R projects. If n units of resource are allocated to project r , then the resulting profit is $p_{r,n} > 0$, for $r = 1, 2, \dots, R$ and $n = 1, 2, \dots, N$. Note that allocating zero units to a project results in zero profit, i.e., $p_{r,0} = 0$ for $r = 1, 2, \dots, R$. An allocation which maximizes the total profit is called an optimal allocation. Our goal is to formulate an $O(N^2R)$ -time algorithm which computes and returns the profit of an optimal allocation. Assume that allocation profits are given in a 2 dimensional array P of size $R \times N$, such that $P[r,n] = p_{r,n}$ for $r = 1, 2, \dots, R$ and $n = 1, 2, \dots, N$.

For example, consider the profit table given below for $R = 3$ and $N = 3$. According to this table, the total profit of an optimal allocation is 10. This is obtained by assigning 1 unit of resource to project 1 (with profit $P[1,1] = 4$), 1 unit of resource to project 2 (with profit $P[2,1] = 1$), and 1 unit of resource to project 3 (with profit $P[3,1] = 5$).

P	0	1	2	N=3
1	0	4	2	6
2	0	1	3	4
3	0	5	5	2

	0	1	2	3
0	0	0	0	0
1	0	4	7	6
2	0	5	8	4
3	0	10	9	2

Clearly show the second and third steps for your dynamic programming formulation:

Step 2) (25 pts) Recursively define the profit of an optimal allocation.

Step 3) (35 pts) Give the pseudocode for an $O(N^2R)$ -time algorithm which computes and returns the profit of an optimal allocation.

Note that you do not have to construct an optimal solution.

① b) ORDER(T, n)
 S keeps the optimal order
 $S[1] \leftarrow T[1]$
 for $i \leftarrow 2$ to n
 if $T[i] > S[i-1]$
 $S[i] \leftarrow T[i]$
 else
 $k \leftarrow i-1$
 while ($T[i] < S[k]$ AND $k \geq 1$)
 $S[k+1] \leftarrow S[k]$
 $k \leftarrow k-1$
 $S[k] \leftarrow T[i]$
 RETURN S

2) For this question I keep two tables, PROFIT and REMAINING. Profit table stores the maximum profit that can be obtained so far. Remaining table stores the remaining resource so far. I calculate my tables in row order.

Let's P_{ij} be the profit of allocating project i, j resource. If $Rem[i, j]$ does not exceed R (total resource) at the time we allocate j to i , we consider it as a candidate allocation. Then we choose the maximum of candidates and update $Profit[i, j]$. The recurrence is as follows:

$$Profit[i, j] = \begin{cases} 0 & \text{if } j=0 \\ \max \left\{ P[i, j], \max_{k=1 \rightarrow N} \left\{ Profit[i-1, k] + P[i, j] \right\} \right\} & \text{if } REM[i-1, k] - k \geq 0 \end{cases}$$

if $i, j > 0$

FIND-OPTIMAL (N, R)

REM is an $R+1 \times N$ matrix ($0 \dots R, 0 \dots N$)

Profit is an $R+1 \times N$ matrix ($0 \dots R, 0 \dots N$)

for $i \leftarrow 0$ to R
for $j \leftarrow 0$ to N
profit[i, j] = 0;

for $j \leftarrow 0$ to N
REM[1, j] = N

for $i \leftarrow 1$ to R
REM[i, N] = 0

initialize

MAX = 0;

for $i \leftarrow 1$ to R

for $j \leftarrow 1$ to N

for $k \leftarrow 1$ to N

if $REM[i-1, k] - k \geq 0$

if $Profit[i-1, k] + P[i, j] > MAX$

MAX = $Profit[i-1, k] + P[i, j]$

REM[i, k] = $REM[i-1, k] - k$

PROFIT[i, j] = MAX

if $P[i, j] > MAX$

MAX = $P[i, j]$

REM[i, j] = $N - j$

PROFIT[i, j] = MAX

(RETURN MAX)

$O(N^2R)$

100

BILKENT UNIVERSITY
Computer Engineering Department
CS 473/573 - Algorithms I
Midweek Exam 6

December 14, 2015
Duration: 90 min.

Name: *İsmail Yıldız*
Student Number: *21201589*

Whenever you are asked to give an algorithm, you are asked to describe it in pseudo-code.

1. (50 pts) You are given a string S of length m and a string T of length n , where $m < n$. Give a dynamic programming formulation for finding the **number** of occurrences of S as distinct subsequences in T . In your solution, clearly give the first three steps of your formulation:

Step 1: Indicate the substructure property (no need for a proof)

Step 2: The recursive definition

Step 3: A bottom-up $\Theta(mn)$ -time **algorithm** that computes and returns the total number of occurrences

Example:

$S = \text{"ABA"}$

$T = \text{"ABABBA"}$

Answer: 6 (i.e., you can form "ABA" in T via 6 different subsequences with index sequences (1,2,3), (1,2,6), (1,4,6), (1,5,6), (3,4,6), (3,5,6))

1- Total number of subsequences up to and including an index satisfying the condition is equal to the total number of subsequences up to but not including that index plus total number of subsequences ending at that index satisfying the condition.

2-
$$c(i,j) = \begin{cases} 0, & \text{if } i=0 \vee j=0 \\ c(i,j-1), & \text{if } i \geq 1 \wedge j > 0 \wedge T[j] \neq S[i] \\ 1 + c(i,j-1), & \text{if } i=1 \wedge j > 0 \wedge T[j] = S[i] \\ c(i-1,j-1) + c(i,j-1), & \text{otherwise} \end{cases}$$

21000025 A.H.	210000696 S.	210000698 R. Yazar	210000700 S.	210000702 S.
Bani KALAFERİA 210000336 F.	Seren ERGÜVEN 211000693 S.	Yekta KURT 211000694 Y.	Elif KAYA 211000360 Y.	Oğuzhan YILMAZ 211000702 S.
Ahmet Safa KAYHAN 210000000		Baran SEZER		

3- NO-OF-APPEARANCES (S, T, m, n)

$C \leftarrow$ empty m by n matrix (zero indexed)

for $i \leftarrow 1$ to m

$C[i][0] = 0$

for $j \leftarrow 1$ to n

$C[0][j] = 0$

$C[1][j] = C[1][j-1]$

if $(S[1] == T[j])$

$C[1][j]++$

for $j \leftarrow 1$ to n

for $i \leftarrow 2$ to m

$C[i][j] = C[i][j-1]$

if $(S[i] == T[j])$

$C[i][j] += C[i-1][j-1]$

return $C[m][n]$

Running time analysis:

$$\Theta(m) + \Theta(n) + \Theta(mn) + \Theta(1) = \Theta(mn)$$

50